

Monorepo CD NDados

Guia do repositório-esqueleto e onboarding do analista recém-alocado. O que é cada pasta, como o padrão se explica sozinho e o que fazer nos primeiros minutos.

Padrão de projetos de Ciência de Dados · distribuição por **cópia independente** · mini-cérebro nativo em **.claude/**

1 O que é este repositório

Este é o **esqueleto canônico** dos projetos de Ciência de Dados do núcleo. Ele não hospeda vários projetos: cada projeto real nasce como uma **cópia independente** deste modelo. A raiz do repositório é o projeto.

A ideia central é o **mini-cérebro nativo**: toda a inteligência de execução — contexto, regras, guias e skills — viaja **dentro do próprio repo**, como arquivos versionados. Qualquer pessoa (ou IA) que clona o repositório recebe junto tudo que precisa para seguir o padrão, sem depender de conhecimento externo. Se algo só funciona com a cabeça de uma pessoa ou um documento fora do repo, está errado.

2 Princípios de design

Mini-cérebro nativo

Contexto, rules e skills moram no repo (hoje consolidados em **.claude/**) e são lidos automaticamente pela IA a cada sessão. A distribuição é a própria cópia.

Núcleo neutro, plataforma opcional

O núcleo do pipeline (config → prepare → build → train → evaluate → orchestrator) não depende de plataforma. Databricks é uma camada opcional, ligada por um toggle. MLflow é sempre obrigatório — o toggle decide só *onde* ele guarda os dados, nunca se ele existe.

Raiz mínima

A raiz é uma vitrine de ponteiros, não um depósito. Só fica na raiz o que tecnicamente não funciona em outro lugar. O critério é requisito técnico, não gosto.

3 Mapa do repositório

O que existe na raiz e para que serve cada coisa. Os itens marcados como **NÚCLEO** são o coração do padrão; **OPCIONAL** pode não existir dependendo do toggle; **TÉCNICO** fica na raiz por exigência de ferramenta.

CLAUDE.md NÚCLEO	A porta de entrada . É lido automaticamente pela IA no início de toda sessão. Aponta para o cérebro de IA em .claude/ e deixa explícito que ler as regras é obrigatório antes de qualquer alteração.
.claude/ NÚCLEO	O cérebro de IA do projeto — reúne o contexto, as rules (o que pode e o que não pode), os guias e as skills/features reutilizáveis, na pasta nativa que o Claude Code reconhece sozinho. Substituiu a antiga pasta ai/ do design inicial. As skills agora vivem aqui como features.
pipeline/ NÚCLEO	O núcleo neutro do pipeline e a anatomia dos modelos. Aqui vivem o contrato de fonte de dados, o carregamento de config, o ponto único de MLflow, os utilitários compartilhados e, por modelo, a anatomia de 5 arquivos: prepare_data , build_model , train , evaluate_model e orchestrator .
platform/ OPCIONAL	A casca do Databricks — bundle, resources, MLProject e o conda gerado. É a parte inteira que o toggle liga ou desliga. Se o projeto for criado sem Databricks, o init apaga esta pasta e o núcleo continua idêntico.
tools/ NÚCLEO	Scripts de apoio. O mais importante é o init.py , o scaffolder embutido que roda uma vez após a cópia: pergunta o nome do projeto e o toggle Databricks, poda o que não é usado e nomeia o primeiro modelo. Também moram aqui os checkadores usados pelas cancelas do CI.
tests/ NÚCLEO	Os testes automatizados (pytest), que rodam localmente e no CI. Cobrem um teste de fumaça (o pacote importa), o pipeline ponta a ponta com dado sintético e o init.py nos dois caminhos do

toggle. São o que transforma uma regra escrita em cancela verificável.

docs/ NÚCLEO	Documentação de referência do esqueleto, incluindo os ADRs (registros das decisões de arquitetura) e o guia narrado de uso.
.github/ TÉCNICO	O CI (GitHub Actions). Fica exatamente aqui porque o Actions só executa deste caminho. É onde as regras viram cancelas automáticas: lint, testes, raiz mínima, manifesto em sync e isolamento de plataforma.
pyproject.toml uv.lock TÉCNICO	A fonte única de dependências (uv), com o lock versionado. O conda do Databricks é gerado a partir daqui, nunca escrito à mão. Ficam na raiz porque o uv só procura projeto instalável ali.
.gitignore TÉCNICO	Ignora artefatos locais (ex.: <code>mlruns/</code> , dados brutos, ambientes, caches). O Git só lê neste caminho exato.
README.md TÉCNICO	A porta de entrada humana — é o que o GitHub renderiza na página do repo. Explica o que é e aponta o analista para o guia de uso.

Sobre a fidelidade deste mapa

A raiz acima é o que existe de fato no repositório. O detalhamento *interno* de cada pasta reflete o design do padrão — vale confirmar contra o conteúdo real (uma árvore `find`) antes de tratar como definitivo, já que o repo evoluiu além do plano registrado.

4 Como as rules funcionam (e por que são confiáveis)

Uma dúvida legítima: as rules em `.claude/` são realmente respeitadas? A cada nova sessão a IA relê as regras? A resposta, segundo a documentação do Claude Code, tem três camadas.

1. Releitura automática — você já tem de graça

Cada sessão começa com o contexto zerado, mas o `CLAUDE.md` é **carregado automaticamente no início de toda sessão**. É esse mecanismo que faz o mini-cérebro nativo funcionar — não é preciso criar uma skill para "forçar a releitura". Aliás, uma skill seria a ferramenta errada: skills carregam sob demanda, por correspondência de descrição, e são *menos* garantidas que o `CLAUDE.md`.

2. Rule referenciada não é rule carregada

Se o `CLAUDE.md` apenas *aponta* ("as regras estão em tal pasta"), a IA só lê essas regras se decidir seguir o ponteiro — não é garantido. Para garantir que entrem no contexto toda vez, há duas saídas confiáveis: usar **@imports** dentro do `CLAUDE.md` (arquivos importados carregam no launch), ou colocar as regras em `.claude/rules/`, que o Claude Code carrega de forma path-scoped (quando a IA mexe em arquivos que casam com a regra).

3. Carregado não é o mesmo que obedecido

A documentação é explícita: esses arquivos são tratados como **contexto, não como configuração forçada**. Para bloquear uma ação independentemente do que a IA decidir, o mecanismo certo é um **hook** (PreToolUse) — a versão local, por sessão, das cancelas de CI. Regras inegociáveis (ex.: "nada importa MLflow fora do ponto único") devem virar hook, não só texto.

Regra prática

Releitura por sessão → já vem no `CLAUDE.md`. Garantir que as rules entrem no contexto → `@import` ou `.claude/rules/`. Tornar uma regra inquebrável → **hook**. Fonte: code.claude.com/docs/en/memory

5 Guia do analista recém-alocado

Você foi alocado num projeto e recebeu este padrão. O objetivo é sair da cópia até o primeiro pipeline rodando sem precisar perguntar nada a ninguém. Em resumo:

Use this template → abrir o repo com a IA → pedir pra IA ler o repo → rodar a skill de kickoff → uv sync → preencher o contexto do projeto → rodar o pipeline

0

Gere sua própria cópia pelo template
No GitHub, use o botão **"Use this template" → Create a new repository**. Isso cria um repositório novo e independente, que é seu. Nunca trabalhe dentro do repo-modelo original.

⚠ **Nunca rode o init no repo-modelo**
O `init` apaga pastas (ex.: `platform/` se você escolher "sem Databricks") e reescreve arquivos. Rodá-lo no modelo mutila o padrão para todo o núcleo. Ele só deve rodar na *sua* cópia.

1

Abra a sua cópia e peça pra IA ler o repo
Abra a cópia num chat/sessão de IA nova. O `CLAUDE.md` já carrega sozinho, mas peça explicitamente para a IA **ler o repositório** — o contexto, as rules e as skills em `.claude/` — para ela absorver o padrão antes de agir.

"Leia este repositório: o CLAUDE.md e tudo em .claude/ (contexto, rules e skills). Depois me diga que padrão você vai seguir."

2

Peça pra IA executar a skill de kickoff
Com o padrão absorvido, mande a IA rodar a **skill de kickoff**. Ela conduz o arranque do projeto: dispara o `init` (nome do projeto, toggle Databricks, nome do primeiro modelo), poda o que não se aplica e deixa a cópia pronta para receber o seu projeto.

"Execute a skill de kickoff para iniciar este projeto."

3

Instale as dependências
Só depois do kickoff, porque o `init` pode ter alterado o `pyproject.toml` (por exemplo, removido o extra do Databricks). Instalar antes traria a lista errada.

uv sync

4

Preencha o contexto do SEU projeto
Diga à IA o que este projeto é: problema de negócio, fontes de dados, stakeholders e glossário do domínio, no arquivo de contexto do projeto dentro de `.claude/`. Enquanto isso não é preenchido, a IA sabe operar o padrão, mas não sabe nada do seu projeto — este passo é o que torna a cópia "sua".

5

Rode o pipeline e comece a trabalhar
Rode o pipeline localmente com o config de exemplo para confirmar que tudo funciona ponta a ponta. A partir daí, os parâmetros ficam em `config`, cada modelo segue a anatomia de 5 arquivos em `pipeline/`, e para adicionar um modelo novo ou uma nova fonte de dados você aciona as skills correspondentes em `.claude/`.

Onde você (e a IA) se orientam
O `CLAUDE.md` na raiz é a porta de entrada e aponta para as rules, os guias e as skills em `.claude/`. Se você está prestes a criar um arquivo solto na raiz, ou fazer algo importar MLflow fora do ponto único, o CI vai barrar — e o motivo está nas rules.